

## Unidad 5: Técnicas de Planificación

### 1. Técnicas de planificación y gestión en proyectos de software

#### 1.1 Gestión de proyectos

Recordemos que la Gestión de Proyectos de Software, es la disciplina que aplica un conjunto de conocimientos, habilidades, herramientas y técnicas a las actividades de un proyecto de software para satisfacer sus requisitos. A diferencia de otros tipos de proyectos, los de software se caracterizan por su alta complejidad, la naturaleza intangible del producto final y la constante evolución de las tecnologías y los requisitos del cliente. El marco esencial en el que se desarrolla esta gestión es el Ciclo de Vida del Desarrollo de Software (SDLC), que define las etapas por las que pasa un producto de software desde su concepción hasta su retiro.

En esta unidad abordaremos el tema de la planificación de las actividades el monitoreo y seguimiento para su correcto cumplimiento.

Antes de abordar la gestión reflexionemos: ¿Que es un proyecto?

Según el Project Management Institute (PMI), la definición más aceptada en el ámbito profesional es: "Un proyecto es un esfuerzo temporal que se lleva a cabo para crear un producto, servicio o resultado único."

Esta definición se basa en las dos características esenciales:

#### *Temporalidad*

- Todo proyecto tiene un principio y un final definidos.
- El final se alcanza cuando:
  - Se logran los objetivos del proyecto.
  - Se determina que los objetivos no se pueden cumplir.
  - La necesidad del proyecto ya no existe.
- Importancia: La temporalidad no implica que el producto o resultado sea temporal (una casa, un software o un puente pueden durar siglos), sino que el esfuerzo para crearlo sí lo es.

#### *Resultado Único*

- El proyecto se emprende para crear algo que nunca antes ha existido en esa configuración, contexto o especificación precisa (Unique Product, Service, or Result).
- Incluso si dos proyectos producen productos similares (ej: dos aplicaciones móviles), la singularidad reside en las diferencias específicas, los interesados (stakeholders), el equipo, la localización, o el tiempo transcurrido.
- Diferencia de la Operación: La operación (o trabajo de negocio rutinario) es continua y repetitiva, produciendo el mismo resultado una y otra vez (ej: la manufactura de autos en una cadena de montaje). El proyecto crea algo nuevo o mejora significativamente algo existente (ej: diseñar y construir el primer modelo de ese auto).

## 1.2 Ciclo de vida del proyecto Vs Ciclo de vida del software

Recordemos las fases del SDLC que guían el proceso de construcción del software de manera organizada. Aunque se usen diferentes las metodologías estas no varían demasiado, sino la forma en que se aplican, repasemos rápidamente fases genéricas:

- a. Planificación: Se define el alcance, los objetivos, el presupuesto, el cronograma y los recursos.
- b. Análisis de Requisitos: Se recopilan, documentan y validan las necesidades del cliente.
- c. Diseño: Se define la arquitectura del sistema, la interfaz de usuario, las bases de datos, etc.
- d. Implementación/Codificación: Se desarrolla el código fuente del software.
- e. Pruebas (Testing): Se verifica que el software cumpla con los requisitos y funcione correctamente.
- f. Despliegue (Deployment): Se instala y se pone en funcionamiento el software para los usuarios.
- g. Mantenimiento: Se realizan correcciones, mejoras y adaptaciones al sistema en producción.

### 1.2.1 Ciclo de Vida del Proyecto de Software (Project Life Cycle)

Existen diferentes referentes en la gestión de proyectos, podemos mencionar algunos como:

- **PMI** Project Managenemt Institute ( <https://www.pmi.org/> ) que define la gestión de proyectos de modo general y aporta muchas herramientas y técnicas de gestión. Este modelos define 5 grupos de porcesos principales para la gestión del proyecto (Inicio, Planificación, Ejecución, Seguimiento y Control, Cierre)
- **CMMI** Capability Maturity Model Integration plantea las acciones para establecer la mejora de los procesos de desarrollo de software, resulta especialmente útil al considerar que la madurez de la organización y la eficiencia de resultados están estrechamente vinculadas (<https://cmmiinstitute.com/>). Tiene un enfoque de mejora continua basado en el desarrollo de habilidades dentro de la organización y la medición del grado de madurez logrado.
- **IEEE 1074** que resulta más específico para la gestión de proyectos de software (<https://ieeexplore.ieee.org/document/1665059>) y define procesos específicos para el monitoreo control, planificación y gestión de proyecto de desarrollo de software (<https://www.ieee.org/>). Este marco define 6 grupos de procesos (Modelado del ciclo de vida, Administración del proyecto, Pre-desarrollo, Desarrollo, Post-Desarrollo y Procesos Integrales)

### 1.2.2 IEEE 1074

Analicemos en particular los grupos de procesos indicados en IEEE 1074 que definen un marco de referencia, no un modelo de ciclo de vida fijo, sino un conjunto de actividades que deben incorporarse en un modelo de ciclo de vida definido en cada proyecto.

1. Modelado del Ciclo de Vida (Life Cycle Modeling): Este grupo es fundamental y único del estándar. Se encarga de definir cómo se llevará a cabo el trabajo. Tiene un proceso clave que es la selección de un **“Modelo del Ciclo de Vida”**. Implica elegir y adaptar un modelo de desarrollo (Cascada, Espiral, Incremental, etc.) a las necesidades y restricciones específicas del proyecto.
2. Administración del Proyecto (Project Management): Se centra en las actividades directivas del proyecto con los siguientes procesos:
  - Iniciación del Proyecto.
  - Supervisión y Control del Proyecto.
  - Gestión de la Calidad del Software.
3. Pre-Desarrollo (Pre-Development): Esta etapa ocurre antes de que comience el desarrollo formal. Se enfoca en la conceptualización y la viabilidad. Incluye:
  - Exploración del Concepto: Se define la idea o necesidad.
  - Asignación al Sistema: Se determina qué parte de la solución será software, hardware o manual.
4. Desarrollo (Development): Este es el grupo central donde se construye el software. Comprende:
  - Requisitos: Definición y análisis de lo que el sistema debe hacer.
  - Diseño: Creación de la arquitectura y el diseño detallado.
  - Implementación: La codificación y las pruebas unitarias.
5. Post-Desarrollo (Post-Development): Se enfoca en la entrega y el soporte del producto una vez que está listo. Procesos:
  - Instalación: Despliegue en el entorno del usuario.
  - Operación y Soporte.
  - Mantenimiento: Corrección de errores y adaptación a cambios.
  - Retiro: Planificación para la eventual discontinuación del software.
6. Procesos Integrales (Integral Processes): Estos procesos son de apoyo y se ejecutan a lo largo de todo el ciclo de vida, no solo en una fase específica como:
  - Verificación y Validación (V&V): Asegurar que el producto se construye correctamente (verificación) y que el producto construido es el correcto para el cliente (validación).
  - Gestión de la Configuración del Software (SCM): Controlar los cambios en los artefactos del proyecto (código, documentos, etc.).
  - Desarrollo de Documentación.
  - Capacitación (Training).

Características:

Como se trata de un marco de referencia, y no es un modelo de ciclo de vida fijo, sino un conjunto de actividades que deben incorporarse en un modelo de ciclo de vida definido para cada proyecto.

Su alcance cubre desde la exploración inicial hasta la retirada del software y en cada adaptación las actividades no incluidas deben ser justificadas.

Exige la aplicación de técnicas de calidad como métricas, verificación y validación.

### 1.2.3 Relación entre Ciclos

Es importante notar que el **Ciclo de Vida del Proyecto** contiene el **Ciclo de Vida del Producto (SDLC)** (requisitos, diseño, codificación, pruebas). La codificación y las pruebas ocurren dentro de las fases de **ejecución del proyecto**.

De esta manera ambos están entrelazados y uno acompaña al otro aportando distintas herramientas o técnicas según el enfoque de la actividad. El objetivo del proyecto incluye el desarrollo de software de un producto (software) de calidad, de forma organizada medible y controlada. El SDLC busca que el producto (software) cumpla con los requisitos y objetivos establecidos.

## 1.3 De la gestión de proyectos

De los modelos o referentes anteriores podemos observar que tienen en común tres grupos de actividades centrales de trabajo:

- **Planificación:** Determinar que se debe hacer, en que tiempo, con qué recursos para cumplir con el objetivo y calidad esperados
- **Monitoreo:** Establecer métricas de seguimiento de las actividades y conocer su grado de cumplimiento respecto del plan.
- **Gestión y Seguimiento:** Tomar las medidas necesarias, realizar los ajustes y correcciones posibles para mantener el proyecto dentro de lo planificado.

Abordaremos en esta unidad las técnicas específicas de estas áreas, pero antes analicemos que pasa con el programador y la gestión.

### 1.3.1 El rol del programador en la gestión del proyecto

El programador es una pieza fundamental que aporta la información necesaria para la toma de decisiones del Gerente de Proyecto bien fundamentadas. Por esto el programador tiene actividades relacionadas con la gestión del proyecto, además de la realización del trabajo específico de escribir código, como:

- **Participar en la Estimación de Tiempos:** La estimación de un proyecto debe ser un esfuerzo colaborativo, y la voz más crítica es la del programador que ejecutará el trabajo (analizamos

estas técnicas detalladamente más adelante). Los programadores son quienes mejor comprenden la complejidad técnica, las posibles dependencias y las "trampas" de la implementación. Una estimación de un gerente sin input técnico suele ser inexacta.

- Participar en la identificación y separación de tareas: En la elaboración de Estructuras de desglose de tareas, descomponiendo los paquetes de trabajo en tareas detalladas.
- Reporta el porcentaje de completitud de sus tareas o, mejor aún, el trabajo restante (enfoque Ágil) en lugar del trabajo consumido.
- Identificación Temprana de Obstáculos (Impedimentos): Su principal responsabilidad de gestión es escalar los problemas de inmediato. Los obstáculos pueden ser técnicos (fallo de infraestructura), de diseño (ambigüedad en el requisito) o de dependencia (necesidad de una API externa).
- Colabora en Reuniones Técnicas y Decisiones Funcionales: Colabora con arquitectos y otros desarrolladores para definir la arquitectura, la elección de tecnologías y los estándares de codificación.
- Aclaración de Requisitos: Trabaja directamente con los Analistas y Propietarios del Producto para aclarar ambigüedades. A menudo, un programador es el primero en identificar una inconsistencia o un requisito no funcional crítico (como el rendimiento o la seguridad) que fue pasado por alto.
- Aprovecha las herramientas de Gestión para organizar su propio trabajo: El uso diligente de las herramientas, mantiene la transparencia y la trazabilidad de los artefactos del proyecto. El programador puede aprovechar las herramientas de gestión del proyecto para su propia organización y reporte de actividades. Utilizando software como Jira, Trello, ClickUp o Azure DevOps para actualizar el estado de sus tareas, asegurando que el estado del proyecto refleje la realidad.
- Vincula las líneas de código (commits) a la tarea o historia de usuario correspondiente. Esto garantiza la trazabilidad desde el requisito inicial hasta el código implementado, un requisito fundamental de la gestión de calidad

### **1.3.2 Importancia del seguimiento continuo y la mejora iterativa.**

Basado en la planificación el seguimiento continuo y el control de avance garantizan el cumplimiento del proyecto.

Por esta razón es fundamental entender diferentes formas de medir y gestionar las actividades. La principal base es contar con indicadores, que brinden a los participantes del proyecto un mecanismo común para conocer el estado de situación. Algunos indicadores básicos que podemos mencionar son:

**Progreso:** muestra cuanto trabajo se ha realizado respecto del total de trabajo planteado el grado de avance, o cumplimiento del alcance establecido.

**Tiempo o atrasos:** relaciona el momento actual con el plan establecido permite identificar que deberíamos tener completado y que falta para completarlo.

Costos relaciona valor estimado de gastos al momento actual, con el gasto real.

Estos tres aspectos representando la triple restricción de los proyectos y tienen impacto en la calidad de la entrega (Alcance, Costo y Tiempo).

El conocimiento de las métricas y su posterior análisis, permite desarrollar un mecanismo de mejora continua de forma iterativa, buscando la corrección y perfeccionamiento de las practica, aportando madurez a la organización como lo plantea CMMI.

Existen diversas técnicas que toman las lecciones aprendidas en cada proyecto, o etapa del proyecto, sean estos grandes acierto o fracasos, los analizan, verifican el contexto de aplicación y buscan la forma de aprovechar esa experiencia en favor del progreso del equipo u organización. Estas lecciones pueden estar relacionadas a la gestión de recursos, a la calidad del producto, a la gestión de los riesgos, la gestión de los stakeholders, a la elección de tecnologías, a la aplicación de metodologías, etc.

### **El enfoque metodológico**

El marco de trabajo de la IEEE establece como primer paso establecer la metodología de trabajo, para ello consideramos que las metodologías predictivas o secuenciales, se basan en una planificación exhaustiva al inicio del proyecto. Se asume que los requisitos pueden ser definidos completamente y que no cambiarán significativamente. El progreso se mide por el cumplimiento de las fases predefinidas. En las metodologías ágiles, que priorizan la respuesta al cambio y el software funcionando antes que la definición exhaustiva, detallada y minuciosa de todos los requisitos y su documentación. El proceso se realiza de manera lineal o iterativa e incremental, según la metodología seleccionada.

En cualquiera de los casos la secuencia de fases del SDLC y las actividades propias de cada una de ellas debe ser planificada, controlada y medida dentro del ciclo de vida del proyecto, esto es la esencia de la gestión de proyectos.

Para establecer un plan la primera etapa es la identificación de las tareas, y la secuencia en que deben realizarse, respetando las fases antes mencionadas, y la característica principal de una tarea dentro de un plan es el tiempo asociado a la misma, el cual también depende de los recursos con los que se ejecute dicha tarea. Una tarea común como escribir una clase o un método de una clase o diseñar las tablas y las relaciones de una base de datos. Dependerá de varios factores, como la tecnología a usar, quienes la realizan (experiencia, capacitación, motivación, etc.), la complejidad misma del trabajo a realizar, y las herramientas o recursos usados.

## **2 Planificación**

Independientemente de la metodología, la gestión eficaz es crucial para el éxito del proyecto. Esta se centra en qué hacer, cuándo y con qué recursos.

En líneas generales podemos pensar en la partición del trabajo en unidades pequeñas y manejables, la estimación o predicción de un tiempo de ejecución, identificar que recursos son

necesarios para completarla y qué condiciones se deben cumplir para considerarla terminada. También se plantea la necesidad de establecer la dependencia entre estas unidades de trabajo para poder identificar qué cosas deben hacerse antes que otras o que actividades deben cumplidas para poder iniciar otras y la determinación del costo asociado.

El siguiente paso es la definición del plan de trabajo, es el momento de jugar con las piezas ordenarlas de la forma más conveniente, respetando sus restricciones, proponiendo cambios, es el momento creativo del líder del proyecto.

Este paso es quizás el de mayor impacto en éxito del proyecto y el más económico (en cuanto a posibilidad de cambio), poder analizar diversas alternativas, dejar espacio para maniobras, prever situaciones críticas, descubrir las tareas de alto impacto, etc. Y todas esas posibilidades de cambio y adecuación del proyecto, estudiadas y analizadas en las etapas tempranas del proyecto, sin haber incurrido en costos aun, permiten elegir las mejores combinaciones de realización, sin el riesgo de perder algo de lo ya hecho. Existe una regla en cuanto a los momentos de los cambios que dice: cuanto más se atrase el cambio más costoso es para el proyecto, de allí que las metodologías ágiles plantean el cambio como parte de la forma de gestión.

## **2.1 Desde las tareas al plan:**

La planificación es el proceso de establecer objetivos, definir las tareas necesarias para alcanzarlos, y estimar el tiempo y los recursos requeridos.

- **Desglose del Trabajo (Work Breakdown Structure - WBS):** Es la descomposición jerárquica orientada a los entregables del trabajo que el equipo debe ejecutar para lograr los objetivos del proyecto. Cada nivel inferior representa una definición más detallada del trabajo. El nivel más bajo son los Paquetes de Trabajo (o tareas).
- **Definición de Tareas:** Cada tarea debe ser clara, medible y tener un responsable asignado (en Ágil, se desglosan en el Sprint Backlog).
- **Estimación de Tiempos y Esfuerzo:** Se utiliza la experiencia y técnicas como la estimación análoga (por proyectos similares) o la estimación paramétrica (cálculos basados en datos históricos). En metodologías ágiles, se usan a menudo Story Points o Tallas de Camiseta para estimar el esfuerzo relativo de los ítems del Product Backlog (historias de usuario).

## **2.2 Tarea**

Una tarea (o actividad) es la unidad de trabajo más pequeña y manejable en la que se descompone un proyecto, necesaria para producir un entregable.

Debe ser:

1. **Clara y Medible:** Debe tener un objetivo bien definido y un resultado que pueda verificarse.
2. **Asignable:** Debe poder asignarse a una persona o equipo responsable.
3. **Con Límites de Tiempo:** Debe tener una duración estimada.



El propósito de la tarea y el enfoque pueden variar de acuerdo a las metodologías que se utilicen según el enfoque sea más del tipo predictivo, en modelos de cascada, modelo V u otros, o más adaptativo con metodologías ágiles.

| Enfoque Metodológico     | Unidad de Desglose Superior                           | Unidad de Trabajo (Tarea)                                                                                                                                                                   | Propósito y Foco                                                                                                                                                        |
|--------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Tradicional (Predictivo) | Paquetes de Trabajo (WBS) o Fases del SDLC.           | <b>Actividad o Tarea de Plan:</b><br>Desarrollar un módulo específico, redactar el documento de requisitos, realizar pruebas de integración.                                                | <b>Cumplir el Plan</b> y finalizar una fase secuencial. La tarea se define en términos de un plan de proyecto fijo.                                                     |
| Ágil (Adaptativo)        | Historias de Usuario (en el <i>Product Backlog</i> ). | <b>Tarea del <i>Sprint Backlog</i>:</b><br>Programar la función de inicio de sesión, diseñar la base de datos para la <i>User Story X</i> , escribir las pruebas unitarias para la Tarea Y. | <b>Entregar Valor</b> en el <i>Incremento</i> del <i>Sprint</i> . La tarea se define en términos del esfuerzo requerido para completar una <b>Historia de Usuario</b> . |

Independientemente del enfoque metodológico la tarea debe cumplir estos criterios de definición de tareas:

- **Objetivo Implícito:** La tarea comienza con un verbo de acción (Diseñar, Implementar, Ejecutar, Desplegar) que define su propósito.
- **Claridad y Asignación:** Se especifica exactamente qué módulo o funcionalidad está involucrada (módulo de inventario, función de login).
- **Medible:** Se usan porcentajes o cantidades exactas que permitan determinar, sin ambigüedad, cuando la tarea está terminada (100% del código, cero bugs, 100% de los métodos de la clase, etc.).
- **Verificable:** Se identifica un entregable tangible (documento aprobado, código en un repositorio, informe de pruebas, registro de la puesta en producción, actividad de despliegue completa, aceptación de una adaptación, cierre de un incidente, etc.) o un estado verificable (deployment exitoso, confirmación de funcionalidad en producción).

### 2.3 Estimación de tiempo de las tareas

La **estimación de tiempos** (o estimación de esfuerzo) en proyectos de software es el proceso de predecir la cantidad de **tiempo de trabajo** necesario para completar una tarea, una funcionalidad, o el proyecto completo. Es un componente crítico de la **planificación de proyectos** y la **gestión de riesgos**.

Esta predicción de tiempo, no puede ser más que una estimación, el tiempo concreto de la tarea será el aquel que dicha tarea efectivamente requiera para su cumplimiento.



Existen posiciones de diferentes actores dentro de la gestión del proyecto que asumen que lo escrito en el plan debe cumplirse sin desvíos que los tiempos estimados se consideran precisos. Pero esto no es totalmente cierto, aun con mucha experiencia, y procesos de mejora que brinden madurez de la organización y del equipo de trabajo, la predicción de tiempo no tiene más rigor que el propio de una estimación. La naturaleza creativa y cognitiva de programación y el concepto de proyecto como actividad única para la construcción o realización de un sistema informático determinan en sí mismos que esta cuantificación sea una aproximación y solo eso. De esto deriva también su criticidad al ser base de estimación de costos, y duración del proceso completo de desarrollo. No por ello debemos escatimar esfuerzos a la hora de realizar la estimación del tiempo y porque de ello depende:

- Establecer Cronogramas Realistas: Determinar cuándo se puede entregar el software.
- Gestión de Recursos: Asignar desarrolladores, testers y otros recursos.
- Presupuesto: El tiempo está directamente ligado al costo del proyecto.
- Expectativas del Cliente/Stakeholders: Comunicar un plazo de entrega.

La estimación de software es inherentemente difícil debido a la naturaleza abstracta y cambiante del desarrollo de software. Es más, un arte basado en la experiencia y el conocimiento que una ciencia exacta.

Aun con las consideraciones anteriores debemos esforzarnos por desarrollar estimaciones sólidas, con la mayor calidad y realismo posible, dado que los desvíos significativos pueden afectar a las partes interesadas con impacto negativo para todo el proyecto, una buena base para ajustar la estimación viene de la mano de seguir estos pasos:

#### 1. Definición del Alcance (Scope)

- Asegúrate de que los requisitos estén lo más claros y detallados posible.
- Si el alcance no está bien definido, la estimación será muy imprecisa (un "cone of uncertainty" - cono de incertidumbre).

#### 2. Descomposición del Trabajo (WBS - Work Breakdown Structure)

- Divide el proyecto en partes más pequeñas y manejables: épicas, características, historias de usuario y, finalmente, tareas de bajo nivel.
- Es más fácil estimar 10 tareas de 4 horas cada una que una sola tarea de 40 horas.

#### 3. Aplicación de la Técnica de Estimación

- Elige la técnica más adecuada según la fase del proyecto, el nivel de detalle disponible y la experiencia del equipo.

#### 4. Revisión y Ajuste (Team Review)

- Las estimaciones deben ser revisadas por el equipo de desarrollo que realizará el trabajo, no solo por el Project Manager. Ellos tienen el conocimiento técnico más preciso.
- Documenta las suposiciones y los riesgos asociados a cada estimación (ej: "Asumimos que el API de terceros estará disponible a tiempo").

#### 5. Añadir Colchón (Contingency/Buffer)

- Siempre agrega un margen de contingencia (buffer) para cubrir imprevistos, bugs no esperados, o cambios menores en el alcance. Este margen debe ser proporcional al riesgo e incertidumbre de la tarea.

#### 6. Seguimiento y Re-estimación

- Una estimación no es inmutable. Debe ser revisada a medida que avanza el proyecto y se obtiene más información real. Esto se llama estimación progresiva o rolling wave planning.

### 2.4 Técnicas de estimación

Las técnicas se pueden clasificar generalmente en empíricas/comparativas y analíticas.

#### A. Técnicas Basadas en la Experiencia (Empíricas/Comparativas)

Estas técnicas se basan en el conocimiento histórico o la opinión experta.

##### 1. Estimación por Analogía

- El estimador compara la nueva tarea (o proyecto) con una tarea similar realizada en el pasado.
- Es útil en las etapas iniciales del proyecto cuando hay pocos detalles, asumiendo que el proyecto es similar en complejidad y tecnología.
- Ejemplo: "Este nuevo módulo de login parece ser un 20% más complejo que el que hicimos en el Proyecto X, que nos tomó 40 horas, así que estimo 48 horas."

##### 2. Estimación por Juicio de Expertos (Delphi)

- Se solicita la opinión de múltiples expertos de forma anónima y se utiliza una ronda de retroalimentación para llegar a un consenso.
- Reduce el sesgo individual y evita que una persona domine la estimación. Es muy útil en proyectos nuevos o con tecnología desconocida.

##### 3. Planning Poker (Estimación Ágil)

- Es una técnica colaborativa donde el equipo utiliza tarjetas numeradas (generalmente la secuencia de Fibonacci: 1, 2, 3, 5, 8, 13, etc.) para votar su estimación para una historia de usuario. Se discuten las diferencias hasta que el equipo converge en una sola estimación.
- Es la técnica estándar en equipos Scrum/Agile. La unidad de medida suele ser "Story Points" (Puntos de Historia), que representan el esfuerzo, la complejidad y el riesgo relativos, no horas.

#### B. Técnicas Basadas en el Cálculo (Analíticas/Paramétricas)

Estas técnicas utilizan fórmulas o modelos basados en métricas.

#### 4. Estimación de Tres Puntos (PERT - Program Evaluation and Review Technique)

- Se calcula un estimado de tiempo más preciso al considerar tres escenarios:
  - o O (Optimista): El mejor escenario posible.
  - o M (Más Probable): El tiempo que se espera que tome.
  - o P (Pesimista): El peor escenario (incluye problemas típicos).
- La estimación final se calcula típicamente como un promedio ponderado (fórmula PERT):

$$E = (O + 4M + P) / 6$$

Útil para tareas críticas o de alto riesgo donde el rango de incertidumbre es grande.

#### 5. Estimación Paramétrica

- Utiliza una relación estadística entre las variables del proyecto (parámetros) para calcular la estimación.
- Requiere datos históricos confiables. Por ejemplo, si un equipo ha promediado 5 horas por CRUD (Crear, Leer, Actualizar, Borrar) en el pasado, y el nuevo proyecto tiene 20 CRUDS, la estimación sería  $20 \times 5 = 100$  horas.

#### 6. COCOMO (Constructive Cost Model)

- Es un modelo algorítmico que estima el esfuerzo, el costo y el cronograma del software. Utiliza variables como el tamaño del código (en líneas de código o Puntos de Función) y un conjunto de factores de costo (complejidad, experiencia del equipo, uso de herramientas, etc.) para calcular la estimación.
- Para proyectos grandes, formales, o cuando se necesita una estimación basada en un modelo matemático robusto.

### 2.4.1 Técnica Juicio de Expertos (Método Delphi)

El objetivo principal es lograr un consenso en la estimación de un grupo de expertos de manera anónima e iterativa, evitando que la opinión de una persona dominante o un sesgo de anclaje influya en el resultado final.

#### Fase 1: Preparación y Selección de Expertos

##### 1. Definir el Objetivo y el Alcance

- Claridad: La tarea o el módulo a estimar debe estar perfectamente definido.
- Información: Se debe proporcionar a los expertos toda la documentación relevante (requisitos, arquitectura propuesta, dependencias, etc.).

##### 2. Seleccionar a los Expertos

- Relevancia: Los expertos deben tener experiencia directa en tecnologías similares, dominios de negocio comparables, o haber realizado proyectos de complejidad similar.
- Número: Generalmente se utiliza un número reducido e impar de expertos (ej: 5 a 7) para facilitar el consenso.
- Neutralidad: Es ideal incluir a personas con diferentes perspectivas (ej: un desarrollador senior, un arquitecto y un líder de proyecto).

##### 3. Elegir un Coordinador/Facilitador

- Esta persona es responsable de recopilar, resumir y distribuir los datos sin modificar las estimaciones ni revelar quién hizo cada una.

#### Fase 2: Primera Ronda de Estimación (Individual y Anónima)

##### 4. Estimación Individual

- El coordinador proporciona a cada experto el paquete de información y un formulario de estimación detallado (puede ser una hoja de cálculo o una herramienta en línea).
- Cada experto realiza su estimación de tiempo de forma independiente y anónima, generalmente en horas o días de esfuerzo.
- Es fundamental que el experto documente sus suposiciones y riesgos clave que justifican su estimación (ej: "Estimé 80 horas asumiendo que el framework se integrará sin problemas").

##### 5. Recopilación y Análisis

- El coordinador recoge todas las estimaciones.
- Se calcula el valor promedio (mediana o media) y se determina el rango de dispersión (la diferencia entre la estimación más alta y la más baja).

Fase 3: Rondas de Retroalimentación y Convergencia (Iterativa)

6. Retroalimentación Anónima

- El coordinador distribuye los resultados al grupo de expertos.
- Esta retroalimentación no incluye la identidad del estimador. Solo muestra:
  - o El rango de estimaciones (mínima y máxima).
  - o El promedio o mediana.
  - o Un resumen anónimo de las justificaciones, suposiciones y riesgos clave, especialmente de los expertos que dieron las estimaciones más distantes (los outliers).

7. Re-estimación y Discusión (Opcional en Delphi Puro)

- Wideband Delphi: En esta variante popular, se realiza una reunión corta y estructurada para que los expertos discutan las razones de la alta dispersión. Los que estimaron muy alto o muy bajo justifican sus posiciones. La regla clave es: no se discuten números, solo las suposiciones y riesgos técnicos.
- Delphi Puro: El proceso es 100% por cuestionario y no hay reunión. Los expertos solo revisan el feedback anónimo por escrito.
- Basándose en la retroalimentación, cada experto realiza una segunda estimación revisada, nuevamente de forma independiente y anónima.

8. Iteración hasta el Consenso

- Las Fases 6 y 7 se repiten (típicamente 2 o 3 rondas) hasta que:
  - o El rango de estimaciones sea lo suficientemente estrecho (ej: la diferencia entre la máxima y la mínima es inferior al 10-15%).
  - o Se ha llegado a un consenso sobre un valor final.

Fase 4: Documentación de la Estimación Final

9. Estimación Final y Cierre

- Una vez alcanzado el consenso, la estimación final se establece como el promedio (o la mediana) de la última ronda.
- Se documenta la estimación final junto con todas las suposiciones y riesgos que el proceso ayudó a clarificar y validar.

Ventajas del Juicio de Expertos

- Reduce Sesgos: El anonimato mitiga la presión social y el riesgo de que el experto junior se alinee con el senior.

- Integra Múltiples Perspectivas: Combina el conocimiento técnico, el de dominio y la experiencia histórica de varias personas.
- Identifica Riesgos: Al pedir justificaciones, el proceso obliga a los expertos a pensar en las suposiciones y los riesgos que pueden afectar el esfuerzo, mejorando la calidad de la planificación.

#### 2.4.2 Técnica Estimación de Tres Puntos

Este método es especialmente valioso cuando se enfrentan proyectos con **alta incertidumbre** o donde faltan datos históricos de proyectos comparables. Se compone de tres pasos

##### Paso 1: Estimación del Esfuerzo (Cifras)

El juicio del experto puede considerado determinando un solo valor de acuerdo a su experiencia personal y el conocimiento de los recursos disponibles, generalmente en organizaciones y grupos pequeños, o también, siguiendo el modelo de **Estimación de Tres Puntos (PERT)**, debe proporcionar tres valores para capturar el rango de incertidumbre.

| Escenario               | Descripción                                                                                                              | Estimación (Horas/Días-Persona)         |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------------------------------|
| <b>Optimista (O)</b>    | Tiempo si todo va perfectamente y sin problemas.                                                                         | <b>40 horas</b> (5 días de trabajo)     |
| <b>Más Probable (M)</b> | Tiempo que se espera que tome, asumiendo problemas menores típicos.                                                      | <b>60 horas</b> (7.5 días de trabajo)   |
| <b>Pesimista (P)</b>    | Tiempo si ocurren los riesgos o problemas conocidos (ej. bugs de integración, retrasos en la documentación de terceros). | <b>100 horas</b> (12.5 días de trabajo) |

La estimación final (ponderada) será calculada por el Coordinador, no por el experto.

##### Paso 2. Justificación y Suposiciones Clave

Aquí es donde el experto explica su razonamiento, lo cual es la parte más valiosa del método Delphi, ya que la diferencia en las cifras a menudo reside en las suposiciones.

| Título                                    | Pregunta                                                                                                    | Respuesta del Experto                                                                                                                                                                      |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Justificación del 'Optimista' (O):</b> | ¿Qué tiene que ocurrir para completar la tarea en el tiempo Optimista?                                      | <i>Asumo que la documentación de Stripe es clara y el entorno de pruebas está disponible desde el día 1. No hay necesidad de refactorizar código existente.</i>                            |
| <b>Justificación del 'Pesimista' (P):</b> | ¿Qué factores podrían hacer que la tarea se extienda hasta el tiempo Pesimista?                             | <i>El mayor riesgo es que la lógica de manejo de errores de la API sea compleja o que tengamos que lidiar con un requisito de cumplimiento de seguridad (PCI) no anticipado.</i>           |
| <b>Suposiciones Críticas:</b>             | ¿Qué hechos estás asumiendo que son ciertos para realizar la estimación? (Ej. disponibilidad, herramientas) | <i>1. El equipo de Backend X habrá finalizado el endpoint de autenticación antes de que comience mi trabajo. 2. La base de datos ya tiene un esquema definido para los tokens de pago.</i> |

### Paso 3. Identificación de Riesgos

| Riesgo                   | Descripción                                                                                        | Impacto Estimado  |
|--------------------------|----------------------------------------------------------------------------------------------------|-------------------|
| <b>R1 (Integración):</b> | La versión del SDK de pagos es incompatible con la versión actual de nuestro framework.            | 16-24 horas extra |
| <b>R2 (Requisitos):</b>  | Faltan requisitos claros sobre el manejo de transacciones fallidas y mensajes de error al usuario. | 8 horas extra     |

Aquí proponemos un ejemplo de una planilla de estimación permite la aplicación del juicio de expertos con tres puntos para trabajar en una etapa de alta incertidumbre:



| Nombre del Proyecto                                       |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|-----------------------------------------------------------|-------------------|---------------------|----------------------------------------------------------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------|----------------------|--------------------------------------------------------------------------------------------|------------------|---------------|
| Fase:                                                     |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| Planilla de Estimación de Tiempos del conjunto de tareas: |                   |                     |                                                                                              |                  |                                                                                                     | Fecha de realización |                                                                                            |                  |               |
| N° Eval/Estimador - ID de planilla                        |                   |                     |                                                                                              |                  |                                                                                                     | Puesto del evaluador |                                                                                            |                  |               |
| ID Tarea                                                  | Descripcion tarea | Tiempo Mas Probable | Escenario Requerido ¿Qué hechos estás asumiendo que son ciertos para realizar la estimación? | Tiempo Pesimista | Escenario Pesimista ¿Qué factores podrían hacer que la tarea se extienda hasta el tiempo Pesimista? | Tiempo Optimista     | Escenario Optimista ¿Qué tiene que ocurrir para completar la tarea en el tiempo Optimista? | Riesgos Posibles | Observaciones |
| T1                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T2                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T3                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T4                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T5                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T6                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T7                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T8                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T9                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| T10                                                       |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|                                                           |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|                                                           |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| Riesgos                                                   | Descripcion       |                     |                                                                                              |                  | Mitigar                                                                                             |                      | Transferir                                                                                 |                  |               |
| R1                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| R2                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| R3                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| R4                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
| R5                                                        |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|                                                           |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|                                                           |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|                                                           |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |
|                                                           |                   |                     |                                                                                              |                  |                                                                                                     |                      |                                                                                            |                  |               |

### 2.4.3 Tecnica Planning Pocker

Planning Poker es la técnica de estimación más popular y efectiva dentro de los marcos de trabajo ágiles (como Scrum).

Preparación (Antes de la Sesión)

#### A. Seleccionar el Equipo de Estimación

El grupo debe incluir a todas las personas que son responsables de implementar la historia o funcionalidad:

- Desarrolladores: Miembros clave del equipo que harán el trabajo.
- Tester/QA: Para considerar el esfuerzo de prueba.
- Scrum Master/Facilitador: Para guiar el proceso y mantener el tiempo.
- Product Owner (PO): No vota, pero está presente para responder preguntas y clarificar los requisitos.

#### B. Elegir la Unidad de Estimación

En Planning Poker, generalmente se usan los Story Points (Puntos de Historia) en lugar de horas.

- Story Point: Es una unidad abstracta que mide el esfuerzo relativo de una historia de usuario, incluyendo: complejidad, volumen de trabajo, incertidumbre/riesgo.

- La Serie: Se utiliza una variación de la Serie de Fibonacci (o números similares): 1, 2, 3, 5, 8, 13, 20, 40, 100. Esto se hace intencionalmente para reflejar que la precisión disminuye a medida que las tareas son más grandes.

#### C. Definir la Historia Base (Referencia)

Antes de comenzar, el equipo debe acordar una historia de usuario pequeña, bien entendida y ya completada, a la que se le asignará un valor (ej: "Crear un botón de guardar" = 2 Story Points). Esta historia base sirve como un ancla o referencia para todas las estimaciones futuras.

#### D. Las "Cartas"

Cada miembro del equipo de estimación (excepto el PO) recibe un juego de cartas con la serie de números elegida (1, 2, 3, 5, 8, 13, etc.).

### 2. Ejecución de la Sesión

#### Paso 1: Presentación de la Historia

- El Product Owner o analista de negocio presenta la historia de usuario a estimar.
- Explica qué quiere el usuario, por qué es valiosa y cuáles son los Criterios de Aceptación (lo que hace que la historia esté "terminada").

#### Paso 2: Preguntas y Clarificación

- El equipo tiene la oportunidad de hacer preguntas al PO para resolver cualquier ambigüedad técnica o de negocio.
- El facilitador (Scrum Master) debe asegurarse de que todos entienden la historia lo suficiente para estimarla.

#### Paso 3: Estimación Silenciosa (El Voto)

- Una vez que se responden las preguntas, cada estimador selecciona en secreto una carta que representa el esfuerzo (Story Points) que él o ella cree que requerirá la historia.
- La carta permanece oculta hasta que todos hayan seleccionado su voto. No se debe hablar ni discutir durante esta fase.

#### Paso 4: Revelación Simultánea

- A la cuenta de tres ("3, 2, 1, ¡Ahora!"), todos los miembros del equipo revelan sus cartas al mismo tiempo.
- Esto es crucial para evitar el anclaje (que la estimación de la primera persona influya en el resto).

#### Paso 5: Discusión de los Valores Extremos

- Si todas las cartas son iguales (o están muy cerca, ej. todos votaron 5 y 8), se logra un consenso y se asigna ese valor a la historia.

- Si hay una dispersión significativa (ej: votos de 3, 5, 13, 40):
  - El facilitador pide al votante que puso el número más bajo (el 3) que explique por qué cree que es tan fácil.
  - Luego, pide al votante que puso el número más alto (el 40) que explique por qué cree que es tan difícil y qué riesgos está considerando.
  - No se discuten los números, sino el razonamiento (suposiciones, riesgos, dependencias técnicas, etc.).

#### Paso 6: Re-votación y Consenso

- Después de la discusión, el equipo es invitado a re-votar la misma historia, aplicando los nuevos conocimientos adquiridos.
- Se repiten los Pasos 3, 4 y 5 hasta que el equipo alcance un consenso o la dispersión sea mínima.
- Consenso: Cuando los votos están muy cerca, el equipo puede acordar utilizar la mediana o, más comúnmente, tomar el voto más alto de los que quedan, ya que suele capturar un riesgo que alguien más pasó por alto.

#### 2.4.4 Técnica COCOMO

COCOMO (CONstructive COSt MOdel) es uno de los modelos de estimación de costos y esfuerzo de software más conocidos y utilizados en la ingeniería de software. Fue desarrollado inicialmente por Barry Boehm en 1981 (COCOMO 81) y luego revisado como COCOMO II para adaptarse a las metodologías y tecnologías modernas (como el desarrollo iterativo y el reuso de software).

COCOMO es un modelo de estimación basado en métricas matemáticas y utiliza el tamaño del producto de software como el principal predictor de esfuerzo.

Es una técnica compleja que requiere un grado de madurez alto y la capacidad definir el tamaño de las piezas en líneas de código. Los equipos con poca experiencia, o que se encuentran incorporando tecnologías nuevas, en general, no tienen posibilidades de estimar adecuadamente el tamaño en líneas de código.

##### Jerarquía de Modelos (COCOMO 81)

El modelo original se presenta en tres niveles de detalle, que se aplican a medida que se tiene más información sobre el proyecto:

- Básico: Ofrece una estimación rápida y aproximada. Utiliza únicamente la estimación de tamaño (Líneas de Código Fuente) y el modo de desarrollo (Orgánico, Semi-Acoplado o Empotrado).
- Intermedio: Incluye 15 atributos de costo (o drivers) para refinar el esfuerzo estimado, considerando factores como la fiabilidad, la experiencia del personal, y las restricciones de hardware.

- Detallado: Incorpora todas las características del modelo intermedio y aplica los atributos de costo a fases específicas del ciclo de vida (Análisis, Diseño, Programación, Pruebas), permitiendo una asignación de personal más precisa.

COCOMO II (El modelo moderno)

La versión moderna se estructura en tres modelos de estimación que se aplican en diferentes etapas del ciclo de vida del proyecto:

1. Composición de Aplicación (Application Composition): Utilizado en las primeras etapas de exploración o prototipado, a menudo emplea Puntos Objeto para la estimación del tamaño.
2. Diseño Temprano (Early Design): Se utiliza una vez que se han estabilizado los requisitos y se ha definido la arquitectura básica. Utiliza Puntos Función o Líneas de Código Fuente Estimadas (KSLOC), junto con 7 Factores de Costo y 5 Factores de Escala.
3. Post-Arquitectura (Post-Architecture): Se aplica durante la construcción y mantenimiento del software. Utiliza KSLOC o Puntos Función y es el modelo más detallado, con 17 Multiplicadores de Esfuerzo y 5 Factores de Escala.

Implementación (Cómo se utiliza)

La implementación se basa en un proceso de cálculo iterativo que ajusta un esfuerzo nominal.

La Fórmula General de Estimación: La base matemática de COCOMO es una ecuación exponencial:

$$\text{Esfuerzo (PM)} = A \times (\text{Tamaño})^B \times \text{EAF}$$

Donde:

- PM (Personas-Meses): El esfuerzo estimado total.
- A y B: Coeficientes que varían según el modo de desarrollo (COCOMO 81) o los Factores de Escala (COCOMO II).
- Tamaño: Estimado en KSLOC (Miles de Líneas de Código Fuente) o métricas equivalentes (Puntos Objeto, Puntos Función).
- EAF (Factor de Ajuste de Esfuerzo): El producto de todos los multiplicadores de esfuerzo (Cost Drivers).

La Fórmula del Tiempo de Desarrollo (Cronograma)

Una vez que se calcula el Esfuerzo (PM), el modelo también proporciona una fórmula para estimar la Duración del Proyecto o Cronograma (TDEV) en meses:

$$TDEV = C \times (\text{Esfuerzo})^D$$

Donde:

- TDEV: El Tiempo de Desarrollo en meses.
- C y D: Coeficientes que dependen del modo de desarrollo, al igual que A y B.
- Esfuerzo: Es el valor de PM calculado con la fórmula de esfuerzo final (el esfuerzo ajustado por el EAF).

**Esta fórmula demuestra el principio de que agregar más personas no siempre reduce el tiempo de desarrollo de forma lineal (Ley de Brooks), ya que el exponente D es siempre menor a 1 (ej. 0.32 a 0.38), lo que implica que el tiempo se escala mucho más lentamente que el esfuerzo. Y el motivo por el cual incluimos la técnica en este documento.**

## 2.5 Desglose de tareas Construcción de la EDT o WBS (Desglose de Tareas)

El Work Breakdown Structure (WBS) o Estructura de Desglose del Trabajo (EDT) es una descomposición jerárquica y orientada a los entregables del trabajo total a realizar. Es el esqueleto del proyecto y es central en la planificación desde cualquier enfoque.

Cómo Construir el WBS

1. Nivel 1: El Proyecto y Entregables Mayores
  - Comience con el nombre del proyecto en la cima.
  - Identifique los entregables principales (e.g., Sistema Web, Documentación de Usuario, Entrenamiento).
2. Nivel 2: Componentes y Fases
  - Descomponga cada entregable mayor en los principales componentes del producto (subsistemas) o las principales fases del SDLC (Requisitos, Diseño, Construcción, Pruebas).
3. Nivel 3 y Sigüientes: Paquetes de Trabajo (Work Packages)
  - Continúe descomponiendo hasta llegar a los Paquetes de Trabajo, que son los entregables del nivel más bajo en el WBS. Un paquete de trabajo aún no es una tarea; es un entregable intermedio claramente definido (e.g., "Módulo de Autenticación Implementado").
4. El Diccionario WBS y Tareas
  - Finalmente, el Paquete de Trabajo se descompone en las tareas específicas, programadas y asignables que deben ejecutarse para producir dicho paquete. La regla general es que la duración de una tarea no debe exceder de 80 horas (dos semanas) o el periodo de reporte del proyecto.

Ejemplo de WBS (Parcial):

1. Proyecto: Sistema de Gestión de Inventario

2. Fase de Construcción
3. Paquete de Trabajo: Módulo de Autenticación
4. Tareas:
  - 4.1. Diseñar el esquema de base de datos para usuarios.
  - 4.2. Implementar la API de registro de usuarios.
  - 4.3. Implementar la interfaz de inicio de sesión.
  - 4.4. Realizar pruebas unitarias de la API de autenticación.

#### El Desglose en Metodologías Ágiles

Aunque el término WBS se asocia con lo tradicional, el concepto de descomposición se aplica en Ágil, pero con una terminología diferente y un enfoque evolutivo:

- Roadmap/Temas (Themes): Alto nivel, cubriendo el panorama del producto (equivalente a las Fases del SDLC).
- Épicas (Epics): Grandes funcionalidades que se descomponen en Historias de Usuario.
- Historias de Usuario (User Stories): Pequeñas descripciones del requisito desde la perspectiva del usuario. Esto es lo que se planifica en el Product Backlog (similar a Paquetes de Trabajo).
- Tareas (Tasks): La descomposición final que realiza el equipo de desarrollo durante el Sprint Planning, indicando el trabajo necesario para completar la User Story (similar a las tareas finales en el WBS).

Independientemente de la elección de metodologías la gestión del proyecto implica la división del trabajo en tareas, establecer la secuencia y dependencia de las mismas. De esta forma la propia naturaleza de cada actividad, determina la forma en que el proyecto se puede desarrollar. Trabajemos con un enfoque tradicional o un enfoque ágil, no podemos probar software que no hemos desarrollado, no podemos escribir código de lo que no fue diseñado, etc. Entonces es necesario determinar que tareas dependen de otras para construir el plan.

#### 2.5.1 Identificación de Secuencias y Dependencias

La secuenciación de tareas es el proceso de identificar y documentar las dependencias lógicas entre las actividades del proyecto. Es vital para construir el cronograma y determinar el Camino Crítico ( el conjunto de tareas que ponen en peligro el cumplimiento del plan)

##### 2.5.1.1 Tipos de Dependencias

Se utiliza el **Método del Diagrama de Precedencia (PDM)** para ilustrar las relaciones de dependencia:

| Tipo de Dependencia         | Descripción                                                                                        | Notación (A precede a B) |
|-----------------------------|----------------------------------------------------------------------------------------------------|--------------------------|
| <b>Fin a Inicio (FI)</b>    | La tarea <b>B</b> no puede comenzar hasta que la tarea <b>A</b> haya finalizado. (El más común).   | $A \rightarrow B$        |
| <b>Inicio a Inicio (II)</b> | La tarea <b>B</b> no puede comenzar hasta que la tarea <b>A</b> haya comenzado.                    | $A \parallel B$          |
| <b>Fin a Fin (FF)</b>       | La tarea <b>B</b> no puede finalizar hasta que la tarea <b>A</b> haya finalizado.                  | $A = B$                  |
| <b>Inicio a Fin (IF)</b>    | La tarea <b>B</b> no puede finalizar hasta que la tarea <b>A</b> haya comenzado. (El menos común). | $A \updownarrow B$       |

#### 2.5.1.2 Determinación de Precedencia

Para determinar qué tarea precede a cuál, aplique el siguiente razonamiento lógico a cada tarea:

- ¿Qué Tareas (o entregables) son necesarias ANTES de que esta Tarea pueda Comenzar?
  - Ejemplo: No se puede Implementar el Código (B) hasta que el Diseño de la Interfaz (A) esté terminado. A (Fin) a B (Inicio).
- ¿Qué Tareas DEBEN Comenzar o Finalizar una vez que esta Tarea Comience o Finalice?
  - Ejemplo: El Entrenamiento del Usuario (B) debería comenzar tan pronto como la Documentación del Usuario (A) comience, aunque la documentación no esté finalizada. A (Inicio) a B (Inicio).

#### 2.5.2 Secuenciación en Enfoques

##### Enfoque Tradicional

- La secuenciación se realiza a nivel de WBS y tareas al inicio del proyecto.
- Se utiliza el diagrama de red del cronograma (basado en PDM) y el Diagrama de Gantt para visualizar la cadena de dependencias y calcular el Camino Crítico.

### Enfoque Ágil

- La secuenciación estricta y de largo plazo es limitada. En su lugar, se prioriza el flujo de valor.
- Priorización (Secuenciación de Valor): El Product Owner ordena el Product Backlog de las Historias de Usuario basándose en el valor y el riesgo (lo que se entregará antes es lo más importante).
- Dependencias Técnicas (Sprint): Dentro de cada Sprint, el equipo sí define las dependencias técnicas de las Tareas del Sprint (e.g., no puedo probar hasta que esté programado). Esta micro-secuenciación se gestiona a través del Tablero Kanban/Scrum, donde el progreso de una tarea hacia la columna "Terminado" depende de la finalización de las tareas anteriores.

## 2.6 Diagramas de Gantt y Seguimiento del Cronograma

El Diagrama de Gantt es una herramienta esencial en la gestión tradicional (aunque también útil en Ágil para la planificación de alto nivel o Roadmaps).

- Descripción: Es un gráfico de barras horizontales que representa el cronograma del proyecto. En el eje vertical se listan las tareas, y en el eje horizontal se representa el tiempo. La longitud de la barra indica la duración de la tarea.
- Utilidad:
  - Visualización: Proporciona una visión clara de la secuencia y el solapamiento de las tareas.
  - Dependencias: Permite identificar las dependencias (una tarea no puede empezar hasta que otra finaliza) y el Camino Crítico (la secuencia de tareas que determina la duración mínima del proyecto).
  - Seguimiento: Se actualiza el progreso (generalmente el porcentaje completado de la barra) para comparar el avance real con el planificado.

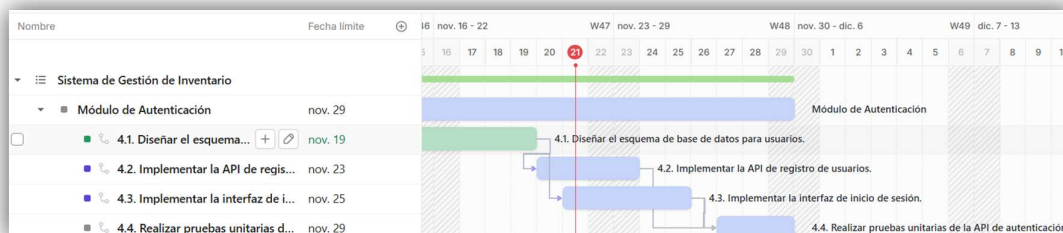


Fig. Diagrama de Gantt

En su forma más simple, un gráfico de Gantt consta de una lista de tareas en el eje vertical, un calendario en el eje horizontal y barras que representan cada tarea. Las flechas representan las dependencias entre las tareas

- Lista de tareas: Detallamos todas las tareas definidas en el WBS en el lado izquierdo del gráfico



- Línea de tiempo: Una línea de tiempo horizontal en la parte superior del gráfico que muestra los días, semanas o meses del proyecto
- Barras: Marcadores horizontales para cada tarea, representando su duración y fechas de inicio y fin
- Flechas: Líneas o flechas que muestran dependencias o tareas que deben ocurrir en un cierto orden

En el diagrama también se pueden incorporar :

- Recursos Asignados: Etiquetas que muestran elementos necesarios y las personas o equipo responsable de completar cada tarea.
- Hitos - Rombos: Iconos especiales que marcan momentos particulares del proyecto (hitos), logros o fechas importantes
- Línea de tiempo actual: Una línea vertical que indica la fecha actual sobre el gráfico.
- Progreso: Un porcentaje o sombreado de barra que muestra el progreso de una tarea
- Camino Crítico: Una secuencia de tareas dependientes críticas para cumplir con el plazo, a menudo la secuencia de tareas más larga.

El diagrama de Gantt resulta especialmente útil para la planificación, monitoreo y seguimiento del proyecto porque:

- Muestra, de un vistazo, todas las tareas involucradas y las desglosa en sub-tareas más pequeñas, puede incluir recursos asignados o las personas directamente responsables de cada tarea.
- Traza un cronograma para todo el proyecto. Esto incluye el tiempo total que tomará terminar todo el proyecto, así como la duración de cada subtarea y facilita ver que se puede ajustar en caso de demoras.
- Mapea dependencias, se puede ver fácilmente las relaciones entre las tareas dependientes y los diferentes recursos asignados.
- Representa el progreso, permiten distinguir el progreso de cada tarea, hitos, y retrasos.

## 2.7 Seguimiento

### 2.7.1 Entregables

Los **entregables** son los productos, servicios o capacidades únicos y verificables que deben producirse para completar un proyecto. En software, estos no solo son el código final, sino también los documentos de las fases previas y los incrementos parciales.

#### Definición de Entregables

- **Enfoque Tradicional:** Los entregables son típicamente documentos formales y el software final (e.g., Documento de Requisitos, Documento de Diseño Arquitectónico, Plan de Pruebas, Software en Producción).
- **Enfoque Ágil:** El entregable clave es el **Incremento de Producto** funcional al final de cada *Sprint*. También existen entregables de proceso (*Sprint Backlog*, *Product Backlog*). La descripción de un entregable en Ágil se basa a menudo en la **Definición de Terminado (Definition of Done - DoD)**, que es un *checklist* de

calidad que todos los elementos deben cumplir para considerarse "terminados" (código revisado, probado, documentado, etc.).

### 2.7.2 Seguimiento y Control

El seguimiento es el proceso de monitorear el progreso del proyecto, identificar desviaciones del plan y tomar acciones correctivas.

- **Enfoque Tradicional:** Se compara el avance real con el Diagrama de Gantt o con la **Línea Base** (el plan aprobado de alcance, cronograma y costo). Se usan informes de estado y mediciones como el **Valor Ganado (Earned Value)** para evaluar el rendimiento.
- **Enfoque Ágil:** El seguimiento es continuo y transparente.
  - **Daily Scrum:** Reunión diaria de 15 minutos para sincronizar al equipo, enfocada en el progreso hacia el **Objetivo del Sprint** y la identificación de **Impedimentos**.
  - **Burndown Charts:** Gráficos que muestran la cantidad de trabajo restante en el *Sprint* o en el proyecto, permitiendo ver si el equipo está a tiempo para terminar.
  - **Velocidad (Velocity):** Métrica que mide cuánto trabajo (en *Story Points*) el equipo puede completar consistentemente en un *Sprint* promedio. Se usa para la planificación futura.
- **Descripción del Entregable para el Cliente:** El entregable debe describirse en términos del **valor** que aporta al cliente, especialmente en Ágil. En la *Sprint Review*, se presenta el Incremento de Producto, explicando qué funcionalidades nuevas se han añadido y cómo resuelven las necesidades del usuario (a menudo descritas como **Historias de Usuario** en formato: "Como [rol], quiero [funcionalidad] para [valor]").

### 2.8 Herramientas para la planificación y gestión de proyectos

Ahora que hemos revisado profundamente las diferentes particularidades de las técnicas veamos como las aplicamos en diferentes herramientas:

Tableros Kamban, en los enfoques ágiles, además de las herramientas como el diagrama de Gantt, se utilizan tableros que facilitan la visualización del trabajo. Están organizados en columna que indican su estado Por Hacer, En Progreso y Terminado. Las actividades están representadas en tarjetas, que se desplazan de una columna a otra, las siguientes imágenes presentan una aproximación ilustrativa de los tableros:

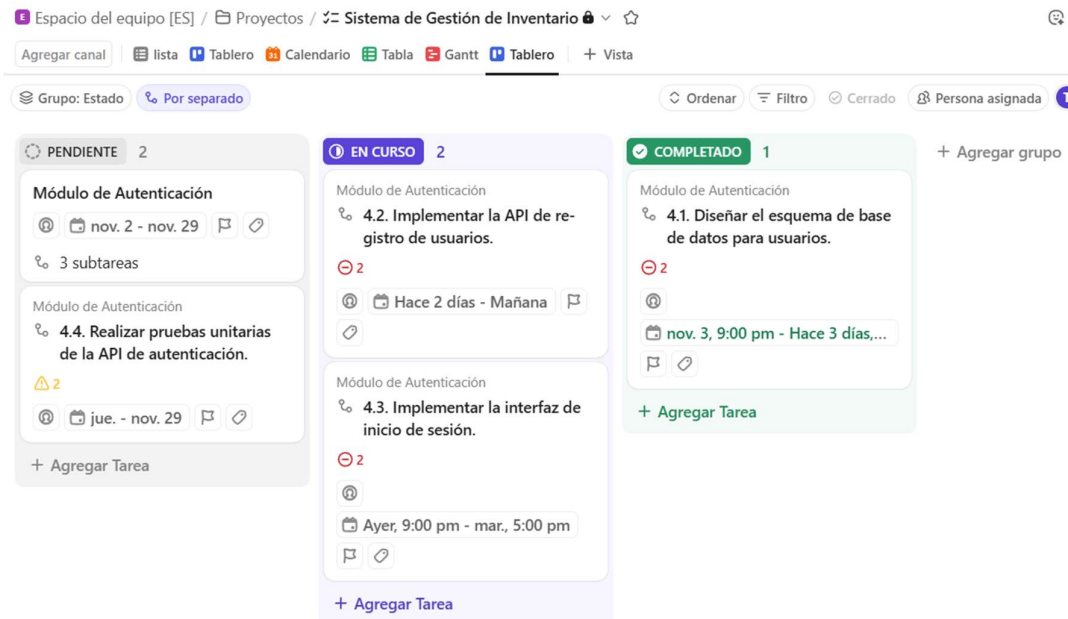


Existen aplicaciones en línea que permiten gestionar esto sin cartulinas y etiquetas adhesivas, además de permitir compartirlos con el resto del equipo.

Un aspecto importante de las herramientas en línea es la posibilidad de asignación a personas o equipos de las actividades y el seguimiento de las tareas.

Es importante considerar que estos tableros no presentan incompatibilidad con las herramientas anteriores, aquí las tareas de una EDT, están en alguna de las columnas, la secuencia o precedencia de tareas determina lo próximo que se debe realizar.

Por ejemplo, aquí tenemos un tablero Kanban de ClickUp a partir del desglose de tareas previo del sistema de gestión de inventario.



### 3. Reflexiones

Aquí incluimos algunas preguntas para pensar en este proceso:

### 3.1 ¿Por qué es útil dividir el proyecto en tareas pequeñas?

Dividir el proyecto en tareas pequeñas, conocido como descomposición del trabajo (la base de la EDT/WBS que mencionamos), es útil por varias razones fundamentales que impactan directamente en el éxito, la predictibilidad y la calidad del proyecto. Porque:

#### 1. Mejora la Precisión de la Estimación y la Predictibilidad

- Reduce la Incertidumbre: Es extremadamente difícil estimar con precisión cuánto tiempo o esfuerzo tomará un componente de trabajo masivo (ej: "Construir Módulo de Pagos"). Es mucho más fácil y preciso estimar tareas cortas y discretas (ej: "Implementar validación de tarjeta de crédito").
- Aplica la Ley de los Números Grandes: Las sobreestimaciones y subestimaciones tienden a cancelarse cuando se suman muchas estimaciones pequeñas, lo que lleva a un pronóstico general del proyecto mucho más fiable.

#### 2. Permite un Mejor Seguimiento y Control

- Visibilidad del Progreso: Con tareas pequeñas, el Gerente de Proyecto y el equipo pueden ver el progreso real de manera granular. Mover una tarea de dos días de "En Curso" a "Terminado" es un hito de progreso claro. Si la tarea dura dos meses, la pregunta "¿cuánto porcentaje llevas?" se vuelve subjetiva e inexacta.
- Identificación Temprana de Riesgos: Las tareas que se estancan o tardan más de lo esperado son señales de alerta tempranas. Si una tarea de un día lleva tres días, sabes que hay un problema que debe abordarse de inmediato, previniendo un gran desastre al final.

#### 3. Facilita la Asignación y la Responsabilidad

- Asignación Clara: Las tareas pequeñas y bien definidas se pueden asignar a una persona o equipo específico con límites claros. Esto elimina la ambigüedad sobre quién es responsable de qué.
- Aumenta la Motivación: Completar tareas pequeñas ofrece una sensación de logro y progreso constante ("pequeñas victorias"), lo cual mantiene alta la moral del equipo.

#### 4. Minimiza el Riesgo y el Impacto de los Cambios

- Aislamiento de Errores: En el desarrollo de software, las tareas pequeñas se traducen en unidades de código más pequeñas. Esto facilita las pruebas unitarias, el aislamiento de bugs y la reducción del riesgo de introducir errores en otras partes del sistema.
- Flexibilidad al Cambio: Si un requisito cambia, es más fácil modificar, descartar o reemplazar una tarea pequeña (que tal vez solo llevó unas pocas horas) que

tener que reestructurar o tirar por la borda semanas de trabajo. Esto es fundamental en metodologías Ágiles.

#### 5. Mejora la Calidad y el Foco

- Permite Revisiones Frecuentes: Las tareas pequeñas permiten integraciones y revisiones de código (code reviews) más frecuentes, lo que garantiza que el trabajo se mantenga alineado con los estándares de calidad del proyecto.
- Foco y Profundidad: El programador puede concentrarse en una sola tarea, logrando un estado de flujo y profundidad de trabajo que resulta en código de mayor calidad, en lugar de intentar hacer malabarismos con una tarea gigante y multifacética.

#### 3.2 ¿Qué pasa si no se estima bien el tiempo de una tarea?

Una mala estimación afecta directamente la base de la gestión del tiempo, el costo del Proyecto y el cumplimiento de los objetivos:

- a) La tarea se estima en X horas, pero en realidad requiere más (o el doble) de tiempo.
- La herramienta de gestión (como Trello o Jira) y el diagrama de Gantt asumen que la tarea se completará a tiempo.
  - Cuando la tarea se retrasa, todas las tareas secuenciales que dependían de ella (dependencias) no pueden empezar, lo que genera un efecto dominó de retrasos.
  - El impacto en los costos del tiempo adicional debe ser gestionado (asumido o trasladado), con la correspondiente
- b) La tarea se estima en Y horas, pero en realidad requiere menos (o la mitad) del tiempo
- Ineficiencia y Tiempo Desperdiciado: Este es el efecto más común, la ley de Parkinson: establece que: "El trabajo se expande hasta llenar el tiempo disponible para su terminación." El tiempo productivo se desperdicia.
  - Aumento Ficticio de la Duración del Proyecto: Las sobreestimaciones, especialmente en la Ruta Crítica del proyecto, inflan artificialmente el cronograma. Esto hace que la fecha de finalización parezca más lejana de lo que realmente necesita ser, lo que resulta en oportunidades perdidas.
  - Costo de Mano de Obra Inflado: Dado que el presupuesto se basa en las estimaciones de tiempo, si las tareas se sobreestiman constantemente, el costo total del proyecto se infla. La organización está pagando por horas que no fueron productivas o necesarias.

- Presupuesto Inexacto: La mala estimación compromete la precisión del presupuesto. La gerencia podría haber asignado esos fondos a otros proyectos con mayor retorno de inversión si la estimación hubiera sido realista.
- Baja Productividad y Retraso en el Valor: Si una tarea crucial se estima en dos semanas, pero podría haberse completado en una, el producto, servicio o característica que genera valor para el cliente se retrasa innecesariamente una semana. En el desarrollo Ágil, la entrega rápida de valor es primordial, y la sobreestimación lo impide.
- Priorización Incorrecta: Si las estimaciones no son fiables, el Gerente de Proyecto puede priorizar incorrectamente las tareas. Podrían dejar una tarea simple y de alto valor para el final porque se sobreestimó como compleja, cuando en realidad podrían haberla entregado rápidamente

### 3.3 ¿Qué ventajas tiene el uso de una herramienta visual como Trello o Kanban?

- Transparencia y Visibilidad Total: Permiten apreciar el estado del proyecto en un vistazo.
- Información Centralizada: Toda la información relevante sobre una tarea está en la propia tarjeta, centralizando el conocimiento del proyecto.
- Comunicación Simplificada: Dado que el tablero es la fuente única de verdad sobre el estado del proyecto, se reduce la necesidad de reuniones constantes para actualizar estados, permite una comunicación asíncrona ya que los comentarios y las actualizaciones se hacen directamente en la tarjeta, manteniendo el contexto de la conversación ligado a la tarea específica.
- Organización Visual: Las listas y la posición de las tarjetas dentro de ellas (normalmente las más importantes arriba) establecen una jerarquía de prioridades clara. El programador siempre sabe qué tarea tiene mayor prioridad para el negocio sin tener que preguntar.
- Reordenamiento ágil : Si las prioridades cambian (algo común en el desarrollo de software), es muy fácil para el Product Owner o Gerente de Proyecto arrastrar y soltar las tarjetas para reordenar el trabajo para la próxima iteración.
- Indicadores o Métricas: Estas herramientas pueden generar métricas clave como el tiempo de ciclo (el tiempo que tarda una tarjeta en ir de principio a fin) y el rendimiento (cuántas tarjetas se completan por semana).
- Datos para la Retrospectiva: Estos datos visuales son esenciales para que el equipo analice qué tan rápido se mueven las tareas e identifique áreas de mejora en sus procesos de trabajo